CSA02 – ASSIGNMENT 1

Grama Jyothi Rural Electricity Co-operative

TASK 4 – Use Macros Safely (4 Marks)

Name: BAVADARANI S Register Number: 1912260562

4.1 Definition of Macros

A macro is defined with #define. Before compilation, the preprocessor replaces macro names with their defined expressions. Macros can be used for repeated calculations such as converting rupees to paise, squaring a value, and finding the maximum of two values.

4.2 C Program Using Safe Macros

```
#include <stdio.h>

#define RUPEES_TO_PAISE(rupees) ((rupees) * 100L)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int main(void)
{
    int a = 4;
    int b = 7;

    printf("Rupees to paise: %ld\n", RUPEES_TO_PAISE(50));
    printf("Square: %d\n", SQUARE(a + 1));
    printf("Maximum: %d\n", MAX(a, b));

    return 0;
}
```

Expected output

```
Rupees to paise: 5000
Square: 25
Maximum: 7
```

Compile and run

```
gcc -std=c11 -Wall -Wextra -o task04 task04.c
./task04
```

4.3 Why Parentheses Matter

Unsafe macro: #define SQUARE(x) x*x. SQUARE(a+1) expands to a+1*a+1. If a is 4, multiplication takes precedence and the result is 9, not 25. The safe definition #define SQUARE(x) ((x) * (x)) correctly groups the argument and the whole expression.

4.4 The Problem with MAX(a, b)

Calling MAX(i++, j) can evaluate i++ twice if that argument is selected: once in the comparison and once in the selected expression. This may change i unexpectedly. Avoid side-effect expressions in macro arguments; a function or inline function is safer.

4.5 Evidence Using gcc -E

```
gcc -E task04.c -o task04_preprocessed.i
gcc -E -P task04.c
```

The -E option runs only the preprocessor. The -P option omits line markers, making the expanded source easier to inspect. For example, SQUARE(a + 1) expands to ((a + 1) * (a + 1)). Save a screenshot of the terminal output.

4.6 When to Use a Function

Use a function for type checking, easier debugging, predictable argument evaluation, and more complex calculations. Macros are useful for simple textual substitutions, but functions are often safer for reusable calculations.

4.7 Alternative Concept: Static Inline Functions

```
#include <stdio.h>

static inline int square(int x)
{
    return x * x;
}

static inline int max_value(int a, int b)
{
    return (a > b) ? a : b;
}

int main(void)
{
    int a = 4;
    int b = 7;

    printf("Square: %d\n", square(a + 1));
    printf("Maximum: %d\n", max_value(a, b));
    return 0;
}
```

Expected output

```
Square: 25
Maximum: 7
```

4.8 Complexity Comparison

Factor	Macro	Static inline function
Execution time	Usually O(1) for these calculations	Usually O(1); may be inlined
Argument evaluation	Can happen more than once	Each argument is evaluated once before the call
Type checking	Limited to expanded expression	Normal C type checking
Code size	Expansion may be repeated at each use	Compiler may reuse or inline implementation
Debugging	Can be difficult	Generally easier

If a macro is used k times, its expanded expression may appear k times in generated code. Actual machine-code size depends on compiler optimisation. A static inline function may reduce repetition, but the compiler is not required to inline it.